

NCHU - [DRL][FR] Final Proposal Phase 2-change03

media.mp4



檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf



PDF

Smart_City_Drone_Law_Enforcement.pdf



PDF

Frank_oral_defense_0725_v7.pptx



簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完整理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明, 也把三個附檔都過了一遍。我的判斷很明確:

最適合你拿來做這門 DRL final project proposal 的題目, 是:

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制 (Adaptive Traffic Signal Control)」。

你附的兩份 drone reference, 其實是在示範同一種教授喜歡的敘事方式: 先把問題的重要性講大, 再整理 related work, 接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**, 還把資料量、模型、限制都量化寫出來; Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文, 但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本, 這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**; 搭配你上傳的 mp4 內容, 我理解教授現在要的不是「先把系統全部做完」, 而是要你先把整條論文故事線講順: **這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚, 就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**; 而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它最貼近教授 **reference** 的味道。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它非常適合 **A-to-Z** 的 **proposal** 寫法。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。🤖 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方向，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based

- RL / DRL traffic signal control : Q-learning、DQN、PPO、multi-agent
- reward / state design : queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment : SUMO / SUMO-RL
- Agent : 先用 DQN 或 PPO
- State : 各車道 queue length、waiting time、current phase、phase elapsed time
- Action : 切換到哪個 phase，或維持 / 切換
- Reward : 負的總等待時間 + queue penalty
- Baselines : fixed-time、actuated control
- Metrics : average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有真實世界動機
- 有明確 baseline 與 metrics
- 有成熟工具鏈
- 有好寫的論文故事

你決定選這題的話，我下一則就直接幫你寫成教授格式的 final project proposal 初稿，包含：題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「這個題目能不能當 final project proposal」：可以。

如果你是問「可不可以直接當 DRL 專案原封不動拿去做」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。  YouTube+2

這個專案本身其實很完整，也很有 proposal 故事性：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。  GitHub+1

但如果你的課是 Deep Reinforcement Learning，那這個 repo 目前不是一個 DRL 專案。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的设计痕跡；從結構看起來，它更像是電腦視覺 + 多模態語音 + 邊緣裝置整合的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。  GitHub+3

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」  GitHub+1

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance
- Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation
- A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你把这个題目改寫成一份符合教授 **A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

另外如果是 MemPalace 這個 AI Agent 永久記憶開源專案呢 是否適合當作題目
且因為 MemPalace 的亮點就是能把 AI 的記憶 "本地化" 如果這個本地化模型還要跑重武器 DRL 有點本末倒置 (本地化就是要越輕量越好)
所以我想從「輕量化 DRL」的切入方向
不要用深度神經網絡 (Deep Q-Network)，改用 Multi-Armed Bandit (MAB, 多臂老虎機) 或 輕量級 Q-Learning

但我連 ai agent 的架設都不會 也沒用過 這樣的程度適合做這個題目嗎
不太適合的話是否有折衷方案 連 ai agent 概念都不用摸呢
顯示更多顯示較少

可以，但我會先給你一個很務實的判斷：

MemPalace 適合當靈感來源，也適合當研究對象；但不適合你現在這個程度直接做成「完整 AI agent 記憶系統實作」的 final project。

因為 MemPalace 本身不是小玩具，它是完整的 **local-first AI memory** 系統：用語意搜尋保留逐字記憶、預設後端是 ChromaDB、知識圖譜用本機 SQLite，還有 MCP 工具、hooks、agent diary、benchmark 等完整配套。官方 repo 也主打本地化、零 API、LongMemEval raw R@5 96.6%。[GitHub+2](#)

你抓的方向其實是對的：**如果題目核心價值是「本地化、輕量化、低依賴」**，那再往上疊一個需要重訓練、重推論的 DQN，確實很可能會讓題目失焦。MemPalace 官方自己也一直強調 bounded wake-up context，大約只載入 L0+L1 的 ~600-900 tokens，且搜尋是按需觸發，不是整包上下文全塞進去；這種設計哲學本來就偏向「資源節制」而不是「深模型硬壓」。這是你可以在 proposal 裡合理主張的切入點。[mempalaceofficial.com+2](#)

但要注意一個關鍵：**MAB 或 tabular Q-learning 並不等於 Deep RL。**

Contextual bandit 比較像是「每回合看到 context，選一個 action，只看到被選 action 的 reward」，是介於 supervised learning 與 RL 之間的問題；tabular Q-learning 則是完整 RL，但它假設你有可管理的有限 state/action，透過探索與更新表格來學策略。官方與教材都把 contextual bandit 和 Q-learning 定義得很清

楚，所以如果你的課名是 *Deep Reinforcement Learning*，你若只做純 MAB，教授有機會質疑「這題不夠 deep」。 

所以我對你這題的建議不是「放棄 MemPalace」，而是：

最適合你的版本

不要做完整 agent。改做：

「本地記憶系統中的輕量化決策層」

也就是把 MemPalace 當成既有記憶後端，你的研究重點不是重造一個 agent，而是研究：

在 token budget、延遲限制、不同 query 類型下，
要用哪一種記憶存取策略最划算？

這樣你的題目就會很乾淨。

我最推薦的題目版本

題目 A：最穩、最適合你

A Contextual Bandit for Token-Budgeted Local Memory Retrieval

白話版就是：

每來一個 query，bandit 根據 context 決定要走哪條 retrieval 路線。

例如 action 可以是：

- 只用 wake-up
- 只用 search
- 先 wake-up 再 search
- 只查某個 wing
- 查 knowledge graph
- top-k 取 3 / 5 / 10

這些 action 都很小、很離散，很符合 contextual bandit「選有限個 action、看 chosen action reward」的型態。MemPalace 官方也提供了不用 agent 的 CLI 與 Python API：你可以直接跑 `mempalace wake-up`、`mempalace search`，或用 `search_memories()` 之類的 Python API 把結果接進你自己的實驗程式，不一定要碰 MCP agent。 

這題最大的優點是：

你完全可以不會架 AI agent，也能做。

第二順位：稍微更像 RL

題目 B：進階但還可控

Tabular Q-Learning for Multi-Step Local Memory Access Policies

這版把問題變成多步決策，例如：

1. 先讀 wake-up
2. 判斷要不要搜 wing
3. 搜不到再深搜
4. 最後決定回傳多少內容

這時候就不是 one-shot bandit，而是有狀態轉移的 sequential decision problem，tabular Q-learning 在概念上比較合理。Q-learning 本來就是不需要先知道完整 MDP，就能從互動資料學 action-value；而且 Gymnasium 也有官方 tabular Q-learning 教學與自訂環境入口，對學生來說比 DQN 友善很多。

 Dive into Deep

Learning+1

但這題比題目 A 稍難，因為你要自己定義 state transition。

你現在的程度，適不適合？

我的判斷是：

做「完整 MemPalace agent + 本地模型 + MCP + hooks + RL」：不適合。

MemPalace 官方文件已經很明白，它平常的使用方式會牽涉到 MCP integration、Claude Code plugin、Gemini CLI、hooks；而「specialist agents」目前官方 docs 主要描述的是透過 MCP tools 寫 diary 的 workflow，不是那種你開箱即用就有完整 agent orchestration 的教學專案。對一個還沒碰過 AI agent 架設的人，這個入口太陡。 mempalaceofficial.com+2

做「MemPalace 當後端 + 你自己寫一個輕量 decision layer」：適合。

因為你可以把所有 agent 概念先拿掉，只保留：

- query
- retrieval action
- reward
- evaluation

這樣本質上就是一個 machine learning / RL 實驗題，不是 agent engineering 題。

如果你想連 agent 概念都不要碰

那我建議你直接再退一步，做成：

題目 C：最保守、最容易成功

Lightweight RL for Local Memory Retrieval Strategy Selection

這題甚至可以**不真的安裝 MemPalace**。

你只要自己準備一個小型資料集，例如：

- 一批對話紀錄 / 筆記 / 文件片段
- 一組 query
- 每個 query 對應的 relevant memory 片段標註

然後模擬幾種 retrieval strategy：

- 全域搜尋
- 分 room 搜尋
- 限 top-k
- 先短上下文再深搜
- 查知識圖譜/metadata 或不查

最後用 contextual bandit 或 tabular Q-learning 去學「哪種 query 應該走哪種策略」，reward 可以是：

reward = recall@k - α ·token_cost - β ·latency

這樣你：

- 不用會 agent
- 不用會 MCP
- 不用接本地 LLM
- 甚至不用碰 MemPalace 真正的工程堆疊

但論文故事仍然成立，而且非常貼近你在意的「本地化」「輕量化」「不要重武器」。這個方向也跟 contextual bandit 在互動式推薦、客服 agent 裡的實際應用邏輯很接近；Microsoft 就有把 contextual bandits / MAB 用在 virtual agent 的推薦與意圖消歧上，重點正是 partial feedback 與資料效率。[arXiv+1](#)

我給你的最終建議

若以你現在的程度，我會這樣排序：

最推薦：

題目 A：Contextual Bandit for Token-Budgeted Local Memory Retrieval

因為最輕、最不需要 agent、最符合你要的本地化邏輯。

次推薦：

題目 C：完全不碰 agent 的 retrieval strategy selection

最保守，成功率最高。

不推薦現在做：

完整 MemPalace agent 系統 + RL 強化

因為你會先死在系統整合，不是死在研究本身。

一句話結論

MemPalace 可以做，但不要把題目定成「做一個 AI agent 永久記憶系統」。

對你最好的版本是把它改成：

「本地記憶系統中，如何用輕量化 RL / contextual bandit，在 token 與延遲限制下選擇最佳 retrieval 策略？」

這樣你幾乎可以**完全不碰 agent 概念**，卻還是保留 MemPalace 的亮點與研究價值。官方 GitHub 就是 MemPalace/mempalace。  [GitHub](#)

下一步最有幫助的做法，是我直接幫你把這題寫成教授 A-to-Z 架構的 final project proposal 題目 + abstract + 方法設計。

好，以下這份就是我幫你整理的 **最佳 prompt**。

它的設計重點，是讓我下一回合直接沿著目前最成熟的交通號誌 DRL 工具鏈與 benchmark 脈絡工作：**SUMO-RL + Gymnasium + Stable-Baselines3 + RESCO**，並且完整對齊你 ilearning 上的交付格式。SUMO-RL 官方明確支援以 SUMO 做 traffic signal control、相容 Gymnasium 與 Stable-Baselines3；RESCO 則是這個領域常見的 benchmark 脈絡。  [Lucas Alegre+3](#)

請你下一回合**直接整段貼上**即可：

你現在要正式開始執行我的「深度強化學習」期末專題 proposal 任務。

一、專案主題

我的 final project 題目已確定為：

****中文題目****

以需求感知深度強化學習進行都市路口自適應交通號誌控制

****英文題目****

Demand-Aware Deep Reinforcement Learning for Adaptive Urban Intersection Traffic Signal Control

二、最重要的執行原則

1. 請先完整理解我先前提供的 ilearning 作業說明、教授 A-to-Z 論文寫作影片、以及我上傳的所有參考檔案，再開始產出內容。
2. 你必須以「完成 ilearning 上要求的 Part A 內容」為核心目標。
3. 你要把自己當成：
 - 論文規劃顧問
 - 專案規格架構師
 - 簡報與口說稿設計師
 - GitHub 提交內容規劃者
4. 你必須明確區分：
 - ****DRL 的 agent**** (state / action / reward / policy / environment)
 - ****AI agent 工具鏈**** (如 AntigraVity、Claude Code 等)
 本任務中可以提到前者，但不要把後者當成研究主題。
5. 你必須優先使用****官方文件、官方 repo、正式論文、benchmark 原始來源****作為主要依據。
6. 你必須主動上網搜尋最新且可信的資料，不可只靠記憶。
7. 除非真的必要，否則不要先問我澄清問題；請先做合理假設並明確寫出假設。
8. 全部回答都用****繁體中文****。
9. 若出現數學符號、RL 符號或較抽象名詞，請主動加上：
 - 符號注釋
 - 白話說明

三、ilearning 任務要求（以我提供的作業截圖為準）

請你依照 ilearning 上的要求處理，尤其注意：

Part A

需要有：

- github
- ppt
- video（錄製約 10 分鐘）

cover following section

1. Introduction / A to F
2. Related Work / NotebookLM
3. Proposed Design / 規格書

被劃掉的部分

- Detail Implementation
- Conclusion

這兩個被劃掉的部分，請**不要當作主要必交章節**，但若你判斷在 proposal 中需要少量提及，可用：

- Future Work
- Expected Implementation Plan
- Expected Outcomes

等較輕的方式處理，不要把它們寫成正式主章節。

Abstract 結構

請明確按照教授要求的這六段邏輯來寫：

1. Attention Getter / Motivation
2. But（挑戰）
3. Cure（我們提出什麼方法）
4. Development（方法設計基於什麼）
5. Experiments（怎麼驗證）
6. Findings（預期或初步可得的發現）

四、你必須先完成的工作順序

請嚴格依照以下順序輸出，不要跳步：

Step 1：先做「作業需求拆解」

請先用條列方式，精準整理：

- 教授真正要交的是什麼
- 哪些是必交
- 哪些不是這次 proposal 的主體
- 我這個題目最應該如何對齊這份作業

Step 2：做「題目定位與研究問題收斂」

請幫我把這個題目收斂成一個適合 proposal 的研究版本，包括：

- 研究背景
- 核心問題
- 研究動機
- 為何值得做
- 與傳統 fixed-time / actuated traffic signal control 的差異
- 我的研究主張（novel angle）

請至少提出：

- 1 個主研究問題（Main Research Question）
- 2~3 個子研究問題（Sub-questions）

Step 3：建立「方法論框架」

請用 DRL 標準語言幫我定義：

- Environment（環境）
- Agent（智能體）

- State (狀態)
- Action (動作)
- Reward (回饋)
- Episode / Horizon
- Training objective

請特別說明：

- 什麼是 demand-aware
- demand-aware 要怎麼體現在 state、reward 或 control logic
- 為什麼這個設計是合理的

Step 4：幫我選擇最適合 proposal 的技術路線

請基於可信來源，幫我規劃：

- 模擬器 / 環境：優先考慮 SUMO / SUMO-RL
- RL 介面：Gymnasium
- 演算法：PP0、DQN 或其他更合理的 baseline / candidate
- 訓練框架：Stable-Baselines3 或其他合適方案
- benchmark / scenario 參考：如 RESCO

請你不要只列名字，要解釋：

- 為什麼這些工具最適合這個題目
- 它們彼此怎麼串起來
- 哪些是 proposal 階段該承諾的
- 哪些是 implementation 階段再處理即可

Step 5：完整寫出 proposal 內文初稿

請直接幫我產出可用於作業的**正式 proposal 初稿**，內容至少包含：

5.1 題目

- 中文題目
- 英文題目

5.2 Abstract

必須嚴格依照 A-to-Z 六段邏輯寫成一個完整 abstract

5.3 Introduction / A-to-F

請把 A-to-F 的邏輯完整展開成 proposal 的導論內容，不要只寫大綱

5.4 Related Work / NotebookLM

這一段要同時滿足兩件事：

1. 真正的 related work
2. 回應教授要求中的「NotebookLM」

請你幫我設計成最合理的寫法，例如：

- 我如何利用 NotebookLM 協助整理文獻
- 但真正 related work 仍要回到正式文獻與 benchmark
- 要避免讓 NotebookLM 看起來像研究主體本身

這一段請包含：

- 傳統交通號誌控制
- RL / DRL traffic signal control
- benchmark 與 simulation 生態
- 我題目的定位

5.5 Proposed Design / 規格書

這一段請用「規格書」思維來寫，包含：

- 系統流程
- 模組拆解
- 輸入 / 輸出
- 交通路口設定假設
- agent interaction flow
- reward design rationale
- baseline design

- evaluation metrics
- 預期實驗情境
- 可能的 ablation study

Step 6：把 proposal 內容轉成「教授會聽得懂、10 分鐘可講完」的簡報結構

請產出一份完整的 PPT 規劃，至少包含：

- 每一頁的頁標題
- 每頁要講的重點
- 建議圖表 / 視覺元素
- 每頁 speaker notes
- 哪些頁適合強調 Attention Getter / But / Cure / Development / Experiments / Findings
- 哪些頁最適合放方法架構圖、state-action-reward 圖、baseline 比較圖

Step 7：產出 10 分鐘口說稿

請幫我寫一份可直接錄影的逐頁口說稿，要求：

- 全長約 9~10 分鐘
- 中文口說自然
- 不要太像念論文
- 但要保有學術感
- 每頁稿都要清楚標註對應投影片頁碼
- 要有開場、轉場、收束
- 結尾不要硬寫 conclusion，可用：
 - proposal summary
 - expected impact
 - future validation

等方式處理

Step 8：規劃 GitHub 交付內容

請幫我設計一個適合 proposal 階段的 GitHub repository 結構，包括：

- README.md
- docs/
- slides/
- proposal/
- refs/
- assets/
- script/
- environment.yml 或 requirements.txt (若合理)
- 其他你認為 proposal 階段應該放的内容

並請直接產出：

1. repo 樹狀結構
2. README.md 初稿
3. 各資料夾應放内容
4. 建議檔名命名規則

Step 9：交付清單總整理

最後請幫我整理成：

- 我可以直接拿去交作業的項目
- 我需要自行補上的最少外部操作
 - 例如：實際上傳 GitHub
 - 實際錄製影片
 - 實際上傳 YouTube
- 一份最終 submission checklist

五、研究内容要求

請務必讓研究内容具備以下特徵：

1. 題目核心必須是：
 - **需求感知深度強化學習**
 - **都市路口**
 - **自適應交通號誌控制**

2. 明確比較：
 - fixed-time control
 - actuated control
 - demand-aware DRL approach
3. 盡量讓 proposal 呈現出：
 - 研究動機強
 - 問題明確
 - baseline 合理
 - 指標具體
 - 系統規格清楚
 - 有 benchmark / toolchain 支撐
 - 適合後續交給 AI coding agent 執行
4. 請不要納入 CityLearn 的任何成分
5. 請不要偏向智慧建築、電池、EV、需求響應
6. 請不要改題目方向
7. 請不要把主體做成「AI agent 系統」
8. 主體必須始終是 DRL traffic signal control

六、資料與來源要求

請你必須同時整合以下來源脈絡：

我已提供給你的資料

- ilearning 作業截圖
- 教授 A-to-Z 影片連結
- media.mp4
- Frank_oral_defense_0725_v7.pptx
- Smart_City_Drone_Law_Enforcement.pdf
- 2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

你要自行補充查找的外部資料

請優先搜尋並引用：

- SUMO 官方文件
- SUMO-RL 官方文件 / 官方 repo
- Gymnasium 官方文件
- Stable-Baselines3 官方文件
- RESCO benchmark
- traffic signal control 相關正式論文
- 必要時可補充 OpenReview、arXiv、官方 GitHub repo

引用品質要求

- 優先用官方與 primary sources
- 不要用低品質部落格當主要依據
- 需要有 citation
- 若是較新的工具或文件，必須查最新資訊
- 不確定的地方要明講，不要硬猜

七、輸出格式要求

1. 請用清楚的 markdown 結構輸出
2. 每個大段落要有標題
3. 提供表格時請真的有比較價值，不要為了表格而表格
4. 有些內容要「可直接拿去交」
5. 有些內容要「可直接複製到 GitHub / PPT / 講稿」
6. 若你能直接產出檔案草稿，也請一併產出
7. 如果內容太長，請你分段清楚輸出，但不要偷省略重要內容

八、最終目標

你這次不是只回答我問題，而是要幫我完成：

- 作業要求解讀
- proposal 主體內容
- related work 方向
- proposed design 規格書
- ppt 章節規劃
- 10 分鐘口說稿
- GitHub 提交內容規劃
- 最終 checklist

請現在開始執行，並先從：

****Step 1：作業需求拆解****
開始。

等你下一回合把這段貼回來後，我就會正式開始執行。

貼上的 Markdown (1).md

檔案

以下先只做你要求的 **Step 1：作業需求拆解**。

我會先把「教授到底要你交什麼」拆乾淨，並把你的題目〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉對齊到這份作業。

Step 1：作業需求拆解

1. 教授真正要交的是什麼

你這次要交的，不是完整研究成品，也不是把系統全部做完；本質上是「一份可被說服、可被展示、可被後續實作的 **proposal package**」。

也就是說，教授要看的是你有沒有把下面這四件事講清楚：

1. 你要解什麼問題
2. 這個問題目前有什麼缺口
3. 你打算怎麼設計方法
4. 你未來打算怎麼驗證它

這和你提供的 A-to-Z 結構完全一致：

Attention Getter / Motivation → But → Cure → Development → Experiments → Findings。

所以這次作業的核心，不是 implementation completeness，而是 **research story completeness**。

2. 必交項目是什麼

依你提供的 ilearning 作業要求，**Part A 必交三個載體**：

- **GitHub**
- **PPT**
- **Video (約 10 分鐘)**

而且內容必須 **cover following sections**：

1. **Introduction / A to F**
2. **Related Work / NotebookLM**
3. **Proposed Design / 規格書**

換句話說，教授不是只要一份 proposal 文字，而是要你把 proposal **做成三種形式的可展示成果**：

- GitHub：作為專案交付與結構化存放空間
- PPT：作為正式報告載體
- Video：作為 10 分鐘口頭簡報證據

所以這份作業的「最終交付物」不是單一文件，而是 **proposal 的完整展示套件**。

3. 哪些不是這次 proposal 的主體

ilearning 上被劃掉的兩部分是：

- **Detail Implementation**
- **Conclusion**

這代表教授已經明示：

這次不要求你進入實作細節，也不要求你做出完整研究結論。

因此這次作業裡，下面這些東西都不應該成為主體：

- 程式碼工程細節
- 訓練超參數表列到極細程度
- 模型已收斂、已 SOTA、已完整驗證的口氣
- 像 final paper 一樣的完整 conclusion

你可以提，但只能用較輕的形式，例如：

- Expected Implementation Plan
- Preliminary Experiment Plan
- Expected Outcomes
- Future Validation

也就是說，這次的正確姿勢不是：

「我已經把系統完整做完，所以來報告結果」

而是：

「我已經把研究問題、方法設計、比較基準、驗證方式規劃完整，足以進入後續實作」

4. 這份作業的評分重心，實際上會落在哪裡

從作業結構看，教授很可能會重看這幾件事：

A. 題目是否夠清楚

不是只講「我要做交通號誌控制」，而是要精準到：

- urban intersection
- adaptive traffic signal control
- demand-aware DRL
- baseline 為 fixed-time / actuated

B. 研究問題是否具體

不是泛泛地說「提升交通效率」，而是要明講：

- 想改善什麼指標
- 與什麼 baseline 比
- 需求感知體現在哪裡
- 為何不是一般 DRL，而是 demand-aware DRL

C. Proposed Design 是否像「規格書」

也就是方法章節不能只講概念，必須有：

- environment
- state
- action
- reward
- scenario
- metrics
- baseline
- experiment plan

D. 10 分鐘簡報是否有完整敘事

你不是在念論文，而是在說服教授：

- 問題值得做
- 工具鏈合理
- 方法設計成形
- 後續可執行

5. 你的題目，怎麼對齊這份作業最漂亮

你的題目其實非常適合這份作業格式，因為它天然就能被拆成教授想看的三大章：

5.1 Introduction / A-to-F 很好寫

這題的動機非常直觀：

都市路口壅塞、固定時制不夠彈性、傳統感應式控制仍有限、尖離峰車流變化大。

這些都能自然對應 A-to-Z 的前半段：

- Attention Getter：城市壅塞與等待成本
- Motivation：交通號誌控制的重要性
- But：fixed-time 與 actuated control 的限制
- Cure：需求感知 DRL
- Development：基於 SUMO / SUMO-RL / Gymnasium / SB3 的設計
- Experiments / Findings：和 baseline 比較 delay、queue、travel time

這條故事線很順。

而且 SUMO-RL 官方本來就是為 **Traffic Signal Control** 提供 RL 環境，並且相容 Gymnasium 與 Stable-Baselines3，這讓你的題目不只是概念上可行，而是工具鏈上也非常對齊。 [lucasalegre.github.io+2](https://github.com/lucasalegre/sumo-rl)

5.2 Related Work / NotebookLM 也很好對齊

這題的文獻脈絡成熟，至少可以分三層：

- 傳統交通號誌控制：fixed-time、actuated
- RL / DRL traffic signal control
- benchmark / simulation 生態：SUMO、SUMO-RL、RESCO

特別是 SUMO 官方已清楚說明 **actuated traffic lights** 是根據交通流或其缺乏而切換，這使你能把傳統方法講得很具體，而不是只空泛地說「舊方法不好」。RESCO 則提供 realistic traffic signal benchmark，讓你在 related work 裡可以談標準化比較，而不只是列 paper。 sumo.dlr.de+1

至於 **NotebookLM**，在這份作業裡最合理的定位不是研究主體，而是：

- 文獻整理輔助工具
- Related work 觀念彙整工具
- 幫你整理 benchmark、baseline、術語差異的閱讀助手

也就是說，這一段應該寫成：

我利用 NotebookLM 輔助整理與歸納文獻，但真正的研究依據仍以官方文件、正式論文與 benchmark 原始來源為主。

這樣最穩。

5.3 Proposed Design / 規格書 更是這題的強項

這題很容易寫成「規格書」而不是空話，因為核心模組很清楚：

- **Environment (環境)**：SUMO / SUMO-RL
- **Agent (智能體)**：DRL traffic signal controller
- **State (狀態)**：queue length、waiting time、current phase、approaching demand
- **Action (動作)**：phase selection / hold / switch
- **Reward (回饋)**：delay、queue、pressure 或其組合
- **Baselines**：fixed-time、actuated
- **Metrics**：average waiting time、queue length、travel time、throughput

SUMO-RL 官方文件明寫 state 和 reward 都容易修改，且可 single-agent / multi-agent；SUMO 官方文件也支援 actuated 與其他交通燈控制機制。這表示你的 proposed design 不只是能寫，還能寫得很像真正要進入實驗。
[lucasalegre.github.io+2](https://github.com/lucasalegre)

6. 你這份作業最應避免的錯誤

這是我目前最重要的提醒。

錯誤 1：把 proposal 寫成 implementation report

不要過早陷入：

- 已完成多少 epochs
- code 細節
- 訓練時間
- 所有超參數表

這次主體不是這個。

錯誤 2：把題目寫太大

你現在的題目應該先聚焦在：

- 都市單一路口
- 需求感知 DRL
- 與 fixed-time / actuated 比較

不要一開始就擴成：

- 大型全城市路網
- 多智能體協同全部路口
- V2X / edge sensing / 智慧城市大平台

proposal 階段要的是**清楚而可執行**，不是大而空。

錯誤 3：把 NotebookLM 寫成研究方法本體

教授要求裡的「Related Work / NotebookLM」比較像是閱讀與整理流程的一部分。
若你把它寫成主研究方法，會偏題。

錯誤 4：把 video 當成只是念投影片

這 10 分鐘其實是 proposal 的口頭版。
你要做的是說服，不是朗讀。

7. 你這個題目目前最正確的作業定位

如果我把它濃縮成一句話，你這份作業最應該被定義成：

一份以 SUMO-RL 為核心工具鏈、以 fixed-time 與 actuated 為 baseline、聚焦單一路口 demand-aware DRL traffic signal control 的研究 proposal 套件。

這個定位之所以好，是因為它同時滿足：

- 研究問題清楚
- baseline 清楚
- 工具鏈成熟
- 簡報容易講
- GitHub 容易整理
- 後續可直接交給 coding agent 落地

8. Step 1 的最終結論

你這次作業真正要交的是：

一套完整的 proposal package，內容主體是：

- Introduction / A-to-F
- Related Work / NotebookLM
- Proposed Design / 規格書

載體則是：

- **GitHub**
- **PPT**
- **10 分鐘影片**

而你的題目

〈以需求感知深度強化學習進行都市路口自適應交通號誌控制〉

非常適合這份作業，因為它能自然對齊：

- 教授要的 A-to-Z 敘事
- 成熟的 benchmark 與工具鏈
- 清楚的 baseline 比較
- 規格書式的方法設計

特別是 SUMO-RL、SUMO 官方 traffic lights 文件、SB3 與 RESCO 這些來源，已經足夠支撐你把 proposal 寫成一個不只是題目好聽，而是真的可被執行的研究計畫。  [lucasalegre.github.io+3](https://github.com/lucasalegre)

下一步我會進入 **Step 2：題目定位與研究問題收斂**。

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

 檔案

Smart_City_Drone_Law_Enforcement.pdf

 PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

 PDF

Frank_oral_defense_0725_v7.pdf

 PDF

From 0 to 1 Deep RL with SMC MDP.pdf

 PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
 2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 **"openspec"**
 3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
- 顯示更多顯示較少

17\ 給 Antigravity 的 change-03 指令草案

 6584113e-92d3-4ec1-a476-cd7666433df0.png

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

附圖以及附檔 是我當初下 Prompt 讓你完成任務的螢幕截圖的時空背景 還有每份檔案 我現在再補上一次 你可以把它當作對這份期末專題的溫故知新

然後直接執行以下

用繁體中文給我「**17. 給 Antigravity 的 change-03 指令草案**」的**正式最佳 Prompt** 不僅僅只是草案 我要可以讓 Antigravity 最精準執行任務的「**最完美且超高品質的最佳 Prompt**」

備註：

因為我之前發生過我跟一位 AI Agent 協作

我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec

我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 官方形容的 SDD 開發模式

他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用

這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

你生給我的最佳 Prompt 要包含 OpenSpec 官方的指令 cli api 等等請 Antigravity 實作

因為是 Antigravity ****開工前最佳 Prompt**** 要有一槌定音的效果

請用最高規格對待 思考縝密 完備 完美執行生成****最佳 Prompt**** 的任務

我要用的是「****官方 OpenSpec 套件****」

顯示更多顯示較少

可以。你這次要的不是一般「草案」，而是**可以直接貼給 Antigravity 開工前使用的正式最高規格 Prompt**。我已把你特別強調的風險寫進去：**必須使用官方 OpenSpec 套件，不准只模擬 OpenSpec workflow，不准自創 SDD 流程，不准手刻假 OpenSpec 結構。**

我也把 OpenSpec 官方 README 中明確提到的安裝與工作方式納入：官方要求 Node.js 20.19+，可用 `npm install -g @fission-ai/openspec@latest` 安裝、在專案目錄執行 `openspec init`，並透過 `/opsx:propose` 建立 change；官方也說 OpenSpec 的價值是讓人與 AI 在寫 code 前先對齊 specs，且每個 change 會有 proposal、specs、design、tasks。 [GitHub+2](#)

本 prompt 也要求 Antigravity 對齊 ns3-gym / Gymnasium / SB3 的介面規格：ns3-gym 官方說它整合 OpenAI Gym 與 ns-3，並列出需要實作 observation space、action space、observation、reward、game over、extra info、execute actions 等介面；Gymnasium 的 Env API 也明確以 `step()` / `reset()`、`action_space` / `observation_space` 作為 environment 基礎；SB3 DQN 文件則顯示 DQN 的 action space 是 Discrete，支撐本 change 的三動作設計。 [GitHub+2](#)

以下請直接複製貼給 Antigravity。

你現在是本專案的 AI coding agent，但目前不是要寫程式，而是要正式執行：

Phase 2 / OpenSpec change 03: opengym-env

你的任務是使用「官方 OpenSpec 套件」建立並填寫 change-03 的規格 artifacts，作為後續 ns3-gym / OpenGym environment 實作的 source of truth。

這不是一般文件整理，也不是你自己模擬 OpenSpec SDD workflow。

你必須使用官方 OpenSpec 套件與官方 OpenSpec workflow。

=====

0. 絕對前提：必須使用官方 OpenSpec，不准自創或模擬

=====

本專案指定使用：

Official OpenSpec package:
@fission-ai/openspec

Official OpenSpec repository:
Fission-AI/OpenSpec

你必須先確認目前專案是否已安裝並初始化官方 OpenSpec。

如果尚未安裝，請使用官方套件安裝：

`npm install -g @fission-ai/openspec@latest`

如果專案尚未初始化 OpenSpec，請在專案根目錄執行：

`openspec init`

如果專案已經有 OpenSpec, 但需要更新 agent instructions 或 slash commands, 請依官方方式執行：

`openspec update`

如果 Antigravity 支援 OpenSpec slash command, 請優先使用官方 OpenSpec slash command 建立本 change：

`/opsx:propose "Create OpenGym environment specification for DRL congestion control over a single bottleneck link"`

如果 slash command 不可用, 仍必須使用官方 OpenSpec 套件初始化後的目錄與規格模式, 建立官方 OpenSpec change artifacts。

嚴格禁止：

- 不准只模擬 OpenSpec workflow。
- 不准自創一套「類 OpenSpec」資料夾。
- 不准說「我用相同概念實作 SDD」。
- 不准手刻一套不受官方 OpenSpec 管理的假規格流程。
- 不准把 OpenSpec 當成純 markdown 命名習慣。
- 不准跳過官方 OpenSpec 套件。
- 不准在沒有確認官方 OpenSpec 狀態前開始建立 artifacts。
- 如果官方 OpenSpec CLI 或 slash commands 無法使用, 必須停止並回報原因, 不得自行改用模擬流程。

你必須在回報中明確列出：

1. 是否已確認官方 OpenSpec 套件存在
2. 是否已確認專案已執行 `openspec init`
3. 是否使用 `/opsx:propose` 或官方 OpenSpec 初始化後的 change 結構
4. 是否完全避免自創 OpenSpec workflow

=====

1. 專案背景與已通過 changes

=====

本專案是 DRL Final Project。

正式題目：

中文：

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文：

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

GitHub repo name：

drl-bottleneck-congestion-control

已驗收通過的 OpenSpec changes：

1. 01-project-charter
2. 02-baseline-benchmark

你必須完整繼承這兩個 change 的內容, 不得自行改題、擴題或跳過。

01-project-charter 已凍結：

- 題目
- scope / non-goals
- toolchain
- baseline list
- metrics
- MVP definition
- downstream change map

02-baseline-benchmark 已凍結：

- NewReno / CUBIC / BBR baseline roles
- single bottleneck topology concept
- throughput / RTT / loss / utility metrics
- scenario matrix
- output artifacts
- benchmark evaluation philosophy
- baseline-first 原則

2. 本次 change 目標

你現在要建立：

OpenSpec change：

03-opengym-env

本 change 的目的，是建立 ns3-gym / OpenGym environment 的規格來源，讓後續 change-04-drl-mvp 可以安全建立 DQN MVP。

你要定義的是 environment interface，不是實作。

本 change 必須定義：

- MDP interface
- environment boundary
- agent boundary
- observation / state space
- discrete action space
- reward function
- episode / horizon
- reset / step conceptual flow
- info dictionary
- random agent smoke test specification
- downstream dependency to 04-drl-mvp
- environment risk register

本 change 必須作為：

- ns-3 simulation 與 RL agent 的橋接規格
- ns3-gym environment 的上游設計規格
- DQN MVP 之前的最後規格閘門
- Antigravity 後續實作前的 source of truth

3. 本階段嚴格禁止事項

本 change 僅允許建立 OpenSpec documentation artifacts。

嚴格禁止：

- 不准寫 C++ code
- 不准寫 Python code
- 不准寫 shell script
- 不准建立 ns-3 topology implementation
- 不准執行 ns-3
- 不准啟動 ns3-gym
- 不准建立 environment wrapper
- 不准執行 random agent
- 不准訓練 DQN
- 不准接 PPO
- 不准建立 PPO artifact
- 不准接 IPFS

- 不准接 QUIC
- 不准修改 Linux kernel TCP
- 不准做 multi-agent
- 不准做 multi-path routing
- 不准做大型 topology
- 不准產生實驗數據
- 不准產生圖表
- 不准宣稱已有 environment 實作結果
- 不准宣稱已有 training 結果

如果你發現需要實作才能回答，請停止並回報「需要另開後續 implementation change」，不要在本 change 中自行實作。

4. 必須建立的 OpenSpec artifacts

請使用官方 OpenSpec change 結構建立或填寫以下 artifacts：

```
openspec/
  changes/
    03-opengym-env/
      proposal.md
      design.md
      tasks.md
    specs/
      mdp-interface.md
      observation-space.md
      action-space.md
      reward-function.md
      episode-step-reset.md
      smoke-test.md
      env-risk-register.md
```

如果官方 OpenSpec 目前產生的資料夾名稱或結構略有不同，請優先遵守官方 OpenSpec 工具產生的結構，但必須確保上述內容都有對應 artifact。

不得把這些文件放在 OpenSpec 外部的自創 docs 目錄，除非官方 OpenSpec 產物同時存在且可追蹤。

5. proposal.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/proposal.md

內容必須包含：

Change 03: OpenGym Environment Specification

Why

說明為什麼在 DQN MVP 前必須先建立 environment specification。

必須明確寫出：

- DQN 不能在 environment interface 未定義前開始
- baseline metrics 必須被 environment 繼承
- observation / action / reward / reset / step 若不先定義，後續 training 不可解釋
- 本 change 是 02-baseline-benchmark 與 04-drl-mvp 的橋樑

What Changes

列出本 change 新增或凍結：

- MDP interface
- environment boundary
- agent boundary

- observation space
- action space
- reward function
- episode / horizon
- reset / step conceptual flow
- info dictionary
- smoke test criteria
- downstream dependency
- environment risk register

What Does Not Change

必須明確寫出：

- 不寫 code
- 不跑 ns-3
- 不啟動 ns3-gym
- 不訓練 DQN
- 不接 PPO
- 不做 IPFS / QUIC / kernel TCP / multi-agent / multi-path
- 不宣稱 environment 或 training 結果

Impact

說明本 change 將約束：

- 04-drl-mvp
- DQN observation input
- DQN action output
- DQN reward signal
- smoke test before training
- baseline-compatible evaluation metrics

Dependencies

必須列出：

- 01-project-charter
- 02-baseline-benchmark

Acceptance Criteria

必須用 checkbox 寫出：

- MDP interface is defined
- environment boundary is defined
- observation space is defined
- action space is discrete
- reward includes throughput / delay / loss
- reset / step conceptual flow is defined
- info dictionary is defined
- random agent smoke test criteria are defined
- downstream dependency to 04-drl-mvp is clear
- no code included
- no training included
- PPO not introduced as MVP
- IPFS / QUIC / multi-agent / multi-path not introduced

6. design.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/design.md

內容必須包含：

Design: OpenGym Environment Interface

Design Goal

定義本 change 的設計目標：

- 將 single bottleneck ns-3 benchmark 抽象為 RL environment
- 讓 DQN MVP 有穩定且可驗收的上游介面
- 讓 baseline metrics 能被 reward 與 evaluation 繼承

Environment Boundary

必須明確寫出：

- environment 是 single bottleneck ns-3 simulation 的 RL abstraction
- agent 不控制整個網路
- agent 只控制 sender-side transmission behavior abstraction
- network dynamics 由 ns-3 simulation 決定

MDP Interface

必須定義：

$M = (S, A, P, R, \gamma)$

並說明：

- S: state / observation space
- A: action space
- P: transition dynamics, 由 ns-3 決定
- R: reward function
- γ : discount factor

Observation Design

Minimal observation 必須包含：

- recent throughput
- delay / RTT signal
- packet loss signal
- congestion indicator
- previous action

Enhanced observation 可列為 future extension：

- queue occupancy
- delay gradient
- throughput moving average
- estimated bottleneck bandwidth
- cross-traffic indicator
- ACK inter-arrival signal

必須明確寫：

- enhanced observation 不得阻塞 MVP
- observation 必須數值化
- observation 必須可 normalization
- feature order 必須在後續 implementation 中明確記錄

Action Design

MVP action space 必須固定為 discrete：

$A = \{\text{decrease, keep, increase}\}$

必須說明：

- decrease: 降低 sender-side transmission behavior
- keep: 維持目前控制強度
- increase: 提高 sender-side transmission behavior

- 不固定實際 cwnd 或 rate multiplier
- 不直接修改 kernel-level TCP
- continuous action 只可作為 future extension

Reward Design

必須使用概念式：

$$r_t = \alpha \cdot \text{normalized_throughput}_t - \beta \cdot \text{normalized_delay}_t - \lambda \cdot \text{normalized_loss}_t$$

注意：

- 使用 λ 表示 loss penalty, 不要用 γ , 避免和 discount factor 混淆
- 不在本 change 固定 α 、 β 、 λ 的數值
- reward 不得只看 throughput
- final evaluation 仍需使用獨立 metrics

Reset / Step Design

Reset concept 必須包含：

- initialize scenario
- initialize flow
- initialize metrics buffer
- initialize previous action
- return initial observation

Step concept 必須包含：

- receive action
- map action to sender-side control abstraction
- advance simulation by one decision interval
- collect metrics
- compute reward
- determine terminated / truncated
- return observation / reward / terminated / truncated / info

Info Dictionary

info 必須包含：

- raw throughput
- raw delay / RTT
- raw loss
- utility
- scenario ID
- step index
- action applied
- baseline-compatible metrics

Smoke Test Design

必須定義 random agent smoke test, 但不得執行。

Smoke test 只驗證：

- reset / step / observation / reward / info flow
- random discrete action accepted
- reward finite
- no crash for fixed number of steps
- info contains required fields

Smoke test 不驗證：

- agent performance
- reward improvement
- baseline outperformance
- DQN training

Downstream Usage

必須明確寫出：

04-drl-mvp 必須使用本 change 的：

- observation spec
- action spec
- reward spec
- episode spec
- info dict spec
- smoke test criteria
- baseline-compatible metrics

04-drl-mvp 不得自行改成：

- continuous action
- PPO MVP
- throughput-only reward
- IPFS / QUIC
- multi-agent / multi-path

Decision Records

至少包含：

- DR-03-01: Define environment before DQN
- DR-03-02: Use minimal observation for MVP
- DR-03-03: Use discrete action for MVP
- DR-03-04: Include delay and loss in reward
- DR-03-05: Require random agent smoke test before training
- DR-03-06: Keep PPO as future extension
- DR-03-07: Do not hand-roll or simulate OpenSpec workflow

7. tasks.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/tasks.md

tasks 必須是規格任務，不是實作任務。

請至少包含以下 checkbox：

- ☐ Verify official OpenSpec package is installed or initialized.
- ☐ Confirm this change is created under official OpenSpec workflow.
- ☐ Confirm dependency on 01-project-charter.
- ☐ Confirm dependency on 02-baseline-benchmark.
- ☐ Confirm no implementation is included in this change.
- ☐ Confirm MDP interface.
- ☐ Confirm environment boundary.
- ☐ Confirm agent boundary.
- ☐ Confirm decision step concept.
- ☐ Confirm episode / horizon concept.
- ☐ Confirm minimal observation fields.
- ☐ Confirm enhanced observation fields.
- ☐ Confirm observation normalization rules.
- ☐ Confirm observation fallback rules.
- ☐ Confirm discrete action space.
- ☐ Confirm action safety rules.
- ☐ Confirm continuous action is future extension only.
- ☐ Confirm reward philosophy.
- ☐ Confirm base reward concept.
- ☐ Confirm reward normalization rules.
- ☐ Confirm reward failure modes.
- ☐ Confirm reward ablation plan.
- ☐ Confirm reset concept.
- ☐ Confirm step concept.
- ☐ Confirm info dictionary requirements.
- ☐ Confirm terminated / truncated concept.
- ☐ Confirm smoke test mission.

- [] Confirm smoke test requirements.
- [] Confirm smoke test non-goals.
- [] Confirm downstream dependency to 04-drl-mvp.
- [] Confirm risk register.
- [] Review that no C++ code is included.
- [] Review that no Python code is included.
- [] Review that no shell command implementation script is included.
- [] Review that no DQN training is included.
- [] Review that PPO is not introduced as MVP.
- [] Review that IPFS / QUIC / multi-agent / multi-path are not introduced.
- [] Submit change-03 to spec owner for review.

8. specs/mdp-interface.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/specs/mdp-interface.md

內容必須包含：

MDP Interface Specification

Purpose

說明此檔案定義本題的 MDP / RL environment interface。

MDP Definition

寫出：

$M = (S, A, P, R, \gamma)$

並解釋每個符號。

Environment Boundary

說明：

- environment 是 single bottleneck ns-3 simulation 的 RL abstraction
- network dynamics 由 ns-3 決定

Agent Boundary

說明：

- agent 只控制 sender-side transmission behavior abstraction
- agent 不控制整個網路
- agent 不直接修改 kernel-level TCP

Transition Assumption

說明：

- $P(s_{t+1} | s_t, a_t)$ 由 ns-3 simulation dynamics 決定
- action 只間接影響下一個 state

9. specs/observation-space.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/specs/observation-space.md

內容必須包含：

Observation Space Specification

Purpose

定義 MVP 與 enhanced observation。

Minimal Observation

必須列出：

- recent throughput
- delay / RTT signal
- packet loss signal
- congestion indicator
- previous action

每個欄位請寫：

- meaning
- expected source
- normalization concern
- MVP required or should-have

Enhanced Observation

列出：

- queue occupancy
- delay gradient
- throughput moving average
- estimated bottleneck bandwidth
- cross-traffic indicator
- ACK inter-arrival signal

並標示：future extension only。

Normalization Rules

寫出：

- all observations must be numeric
- all observations must be normalizable
- feature order must be documented
- values must be bounded or scaled where possible

Fallback Rules

寫出：

- if RTT trace is unavailable, use delay-related statistic
- if queue occupancy is unavailable, use delay / loss proxy
- if estimated bandwidth is unavailable, exclude it from MVP
- if previous action complicates interface, mark it should-have

10. specs/action-space.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/specs/action-space.md

內容必須包含：

Action Space Specification

Purpose

定義 MVP discrete action space。

Discrete Action Set

A = {decrease, keep, increase}

說明每個 action。

Action Safety Rules

必須包含：

- no negative rate / cwnd-like variable
- bounded action effect
- action effect must be logged
- no kernel-level TCP modification
- maintain abstraction layer

Future Continuous Action Rules

寫出：

- continuous action is future extension only
- continuous action requires a separate OpenSpec change
- PPO cannot be introduced in this MVP change

=====

11. specs/reward-function.md 要求

=====

請建立或填寫：

openspec/changes/03-opengym-env/specs/reward-function.md

內容必須包含：

Reward Function Specification

Purpose

定義 reward philosophy 與 base reward concept。

Base Reward Concept

$$r_t = \alpha \cdot \text{normalized_throughput}_t - \beta \cdot \text{normalized_delay}_t - \lambda \cdot \text{normalized_loss}_t$$

並解釋：

- r_t
- α
- β
- λ
- $\text{normalized_throughput}_t$
- $\text{normalized_delay}_t$
- normalized_loss_t

Normalization Philosophy

寫出：

- throughput / delay / loss scales differ
- normalization is required
- weights are not fixed in this change
- 04-drl-mvp may define initial weights

Failure Modes

列出：

- throughput-only reward
- over-penalized delay
- over-penalized loss
- unstable scale
- sparse reward

Future Ablation

列出：

- throughput-only
- throughput + delay
- throughput + delay + loss

12. specs/episode-step-reset.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/specs/episode-step-reset.md

內容必須包含：

Episode / Step / Reset Specification

Purpose

定義 episode、reset、step、terminated / truncated、info dictionary。

Reset Concept

列出 reset 概念流程。

Step Concept

列出 step 概念流程。

Terminated / Truncated Concept

說明 episode ending 與 time-limit / fatal-error 類情境。

Info Dictionary Requirements

必須包含：

- raw throughput
- raw delay / RTT
- raw loss
- utility
- scenario ID
- step index
- action applied
- baseline-compatible metrics

13. specs/smoke-test.md 要求

請建立或填寫：

openspec/changes/03-opengym-env/specs/smoke-test.md

內容必須包含：

Smoke Test Specification

Purpose

說明 smoke test 只驗證 environment flow, 不訓練 agent。

Smoke Test Requirements

必須包含：

- reset returns valid observation
- random discrete action accepted
- step returns next observation
- reward is finite number
- terminated / truncated defined

- info contains required fields
- no crash for fixed number of steps
- logs compatible with change-02 metrics
- observation feature order documented
- action applied recorded

Smoke Test Non-Goals

必須包含：

- no performance expectation
- no reward improvement expectation
- no baseline outperformance
- no final figure
- no DQN training
- no PPO

Acceptance Criteria

列出 smoke test 規格層級驗收條件。

=====

14. specs/env-risk-register.md 要求

=====

請建立或填寫：

openspec/changes/03-opengym-env/specs/env-risk-register.md

至少包含以下風險：

1. observation 無法從 ns-3 直接取得
2. RTT trace 不易取得
3. queue occupancy 不易取得
4. action effect 難以映射到 sender-side control
5. reward scale 不穩
6. throughput-only reward 誘導錯誤策略
7. step interval 不一致導致 baseline / DRL 無法比較
8. random agent smoke test 無法通過
9. Antigravity 直接寫 DQN training
10. PPO 被提前導入
11. environment spec 過度綁死 implementation
12. info dict 缺少 baseline-compatible metrics
13. Antigravity 未使用官方 OpenSpec, 而是模擬 workflow

表格欄位：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

=====

15. 完成後必須回報的內容

=====

完成後請回報以下內容：

1. 官方 OpenSpec 使用確認
 - 是否已安裝 @fission-ai/openspec
 - 是否已執行 openspec init
 - 是否已使用 /opsx:propose 或官方 OpenSpec change structure
 - 是否沒有自創或模擬 OpenSpec workflow

2. 已建立或更新的 artifacts 清單

3. 本 change 的核心決策摘要

- MDP interface
- observation
- action
- reward
- reset / step
- smoke test

4. 明確聲明未做事項

- 未寫 code
- 未跑 ns-3
- 未啟動 ns3-gym
- 未訓練 DQN
- 未導入 PPO
- 未加入 IPFS / QUIC / multi-agent / multi-path

5. 等待 spec owner 驗收

16. 最終驗收標準

本 change 只有在以下條件全部滿足時，才算完成：

- 使用官方 OpenSpec 套件或官方 OpenSpec 初始化後的結構
- 未模擬 OpenSpec workflow
- 未自創 SDD 系統
- 所有 artifacts 位於 OpenSpec change-03 之下
- proposal.md 完整
- design.md 完整
- tasks.md 完整
- specs/mdp-interface.md 完整
- specs/observation-space.md 完整
- specs/action-space.md 完整
- specs/reward-function.md 完整
- specs/episode-step-reset.md 完整
- specs/smoke-test.md 完整
- specs/env-risk-register.md 完整
- no implementation code
- no ns-3 execution
- no ns3-gym launch
- no DQN training
- no PPO MVP
- no IPFS / QUIC / multi-agent / multi-path scope drift
- 完成後停止，等待 spec owner 驗收

這版已經把你最在意的點鎖住了：**Antigravity 必須真的使用官方 OpenSpec 套件與官方 OpenSpec workflow，不准只是模仿 OpenSpec 的資料夾或 SDD 名詞。**

你可以直接拿這段貼給 Antigravity。

我已驗收 commit ae2e535597e2088256117ff408b39791c75de14f。

結論：

Change 02：ns3-baseline-benchmark 的 proposal / design / specs / tasks revisions 已通過驗收。

OpenSpec status 與 strict validation 也已通過：

- isComplete: true
- 4/4 artifacts status: done

- 6 spec files recognized
- openspec validate --strict --json → valid: true, issues: [], passed: 1, failed: 0

因此，Change 02 specification phase 可視為完成。

但目前不批准執行 Change 02 的 /opsx:apply。

原因：

本專案目前仍處於 Phase 2 「用 OpenSpec 建立專案真相來源」階段。

Phase 2 的目標是完成 change-01 / change-02 / change-03 / change-04 的 proposal / design / specs / tasks。

真正的 implementation / ns-3 baseline coding / benchmark execution，應保留到 Phase 3 「Baseline 先行」再開始。

請將 tasks.md 0.11 保持為未完成或標註為：

Spec Owner approval for Change 02 specification is granted, but /opsx:apply is deferred until Phase 3.

正式批准：

Approved to proceed to Phase 2 change-03 : opengym-env proposal.

不批准：

- Do not run /opsx:apply for Change 02 yet.
- Do not start ns-3 coding.
- Do not execute benchmark.
- Do not create experiment results.
- Do not write analysis scripts.
- Do not start ns3-gym implementation yet.
- Do not start DQN / PPO.

下一步請開始：

Phase 2 / OpenSpec change-03 : opengym-env proposal / design / specs / tasks only.

<https://github.com/donuop35/ns3-drl-congestion-control/commit/ae2e535597e2088256117ff408b39791c75de14f>

分割線以上是我請 Antigravity 做完 Phase 2 Change02 後 我再請 Chatgpt 用上方這個 github repo 的 diff 幫我驗收標準後 他給我的要餵回去給 Antigravity 對齊需求的 Prompt

我想要同你的 Phase 2 Change03 最佳 Prompt 一同餵給 Antigravity 一併執行任務

請幫我判斷分割線上的 Prompt 應該安插在你 **Phase 2 Change03 最佳 Prompt** 的哪個段落最適合

或者你認為要合併兩者就需要重新規畫你的 **Phase 2 Change03 最佳 Prompt** 也跟我說 並且告訴我你的判斷準則

不然 我最希望的是直接把兩份 Prompt **直接合併（但我必須知道放在哪個位置，目前我沒有頭緒，我要你幫我判斷分析）**就好
顯示更多顯示較少

我的判斷：**不用重新規劃整份 Phase 2 Change03 最佳 Prompt**，但你這段「Change02 驗收結果 + 不批准 /opsx:apply + 批准進入 Change03」**一定要合併進去**，而且不能放在最後。

最適合的位置是：

放在 Phase 2 Change03 最佳 Prompt 的「0. 絕對前提：必須使用官方 OpenSpec」之後、「1. 專案背景與已通過 changes」之前。

也就是新增一段：

0.5 Phase Gate : Change 02 specification approved, apply deferred

原因很明確：你這段不是普通背景資訊，而是**流程閘門 / approval state / 禁止 apply 的治理指令**。OpenSpec 官方 workflow 是先 /opsx:propose 產出 change artifacts，後續才 /opsx:apply 進入 implementation；官方 README 也把 proposal、specs、design、tasks 與 apply 分成前後階段，並強調「code 前先對齊 specs」。所以你這段應該放在所有任務內容之前，讓 Antigravity 一開始就知道：**Change02 specification 已過，但 apply 被明確延後到 Phase3**。 [GitHub](#)

判斷準則

我用這幾個準則判斷插入位置：

判斷準則	結論
它是什麼類型的內容？	不是任務內容，是 phase gate / approval state
它影響什麼？	影響 Antigravity 是否能執行 /opsx:apply、是否能開始 implementation
它應該在任務前還是任務後？	一定要在任務前，否則 Antigravity 可能先執行錯誤流程
它屬於 Change03 spec 本體嗎？	不屬於 observation/action/reward，而是 Change03 的前置狀態
是否需要重寫整份 Prompt？	不需要，只要新增一個高優先級 Phase Gate 區塊
是否可直接兩份 Prompt 串在一起？	不建議直接亂接，應插入到「官方 OpenSpec 前提」之後

最佳插入位置

你原本的 Change03 最佳 Prompt 開頭大概是：

```
0. 絕對前提：必須使用官方 OpenSpec，不准自創或模擬
...
1. 專案背景與已通過 changes
...
```

請插在中間：

```
0. 絕對前提：必須使用官方 OpenSpec，不准自創或模擬
...

0.5 Phase Gate: Change 02 specification approved, apply deferred
...

1. 專案背景與已通過 changes
...
```

這是最準的位置。

可直接插入的合併段落

你可以把下面這段，直接插進我上一版 **Phase 2 Change03 最佳 Prompt** 的第 0 節後面。

```
=====
0.5 Phase Gate: Change 02 specification approved, apply deferred
=====

Before starting Change 03, you must acknowledge the current OpenSpec phase gate.

The spec owner has reviewed and approved commit:

ae2e535597e2088256117ff408b39791c75de14f

Repository:

https://github.com/donuop35/ns3-drl-congestion-
control/commit/ae2e535597e2088256117ff408b39791c75de14f

The review conclusion is:

Change 02: ns3-baseline-benchmark / baseline-benchmark specification phase has passed review.

The following are confirmed:

- proposal.md approved
- design.md approved
- specs approved
- tasks.md revisions approved
- OpenSpec status passed
- isComplete: true
- 4/4 artifacts status: done
- 6 spec files recognized
- openspec validate --strict --json passed
- validation result: valid: true, issues: [], passed: 1, failed: 0

However, the spec owner explicitly does NOT approve running /opsx:apply for Change 02 at this
time.

Reason:

This project is still in Phase 2: establishing OpenSpec as the project source of truth.

Phase 2 goal:

Complete the proposal / design / specs / tasks for:

- change-01-project-charter
- change-02-baseline-benchmark
- change-03-opengym-env
- change-04-drl-mvp

Implementation must be deferred to Phase 3: Baseline First.
```


Therefore:

- Do not run /opsx:apply for Change 02.
- Do not start ns-3 coding.
- Do not execute benchmark.
- Do not create experiment results.
- Do not write analysis scripts.
- Do not start ns3-gym implementation.
- Do not start DQN.
- Do not start PP0.

Please keep tasks.md item 0.11 unfinished or annotate it as:

Spec Owner approval for Change 02 specification is granted, but /opsx:apply is deferred until Phase 3.

The spec owner formally approves proceeding only to:

Phase 2 / OpenSpec change-03:opengym-env proposal / design / specs / tasks

You are now approved to create Change 03 specification artifacts only.

You are not approved to apply Change 02.
 You are not approved to implement Change 02.
 You are not approved to implement Change 03.
 You are not approved to run any benchmark or training.

If any OpenSpec workflow asks whether to proceed with implementation, you must stop and report back to the spec owner.

不建議放的位置

不要放在以下位置：

1. 不要放在 Prompt 最後。

放最後會太晚，Antigravity 可能已經依前面任務進入建立或 apply 流程。

2. 不要放在「17. 給 Antigravity 的指令草案」後面。

那段是收尾指令，不適合放高優先級流程閘門。

3. 不要放在「已驗收前置內容」裡面混成普通 bullet。

它不是普通「已完成 changes」資訊，而是**批准與禁止事項並存的 governance decision**。

4. 不要放在 tasks.md 要求裡。

tasks.md 是文件任務清單；你這段是整體工作權限，不只是任務項目。

我建議的最終合併方式

你的最終 Change03 Prompt 開頭應該長這樣：

你現在是本專案的 AI coding agent，但目前不是要寫程式，而是要正式執行：

Phase 2 / OpenSpec change 03:opengym-env

你的任務是使用「官方 OpenSpec 套件」建立並填寫 change-03 的規格 artifacts，作為後續 ns3-gym / OpenGym environment 實作的 source of truth。

這不是一般文件整理，也不是你自己模擬 OpenSpec SDD workflow。
 你必須使用官方 OpenSpec 套件與官方 OpenSpec workflow。

=====

0. 絕對前提：必須使用官方 OpenSpec，不准自創或模擬

=====
...原本第 0 節內容...

=====
0.5 Phase Gate : Change 02 specification approved, apply deferred
=====

...插入你這次提供的 Change02 驗收 / defer apply 段落...

=====
1. 專案背景與已通過 changes
=====

...原本第 1 節內容...

這樣是最穩的。

它既保留我原本 Change03 Prompt 的完整度，又把你最新的 GitHub commit 驗收結果與 **/opsx:apply** 禁止條件提升到最高優先級。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/ce914a768d0766ca7d62e18343b5d4f5636182fe>

Antigravity 做完 Phase 2 Change03 ****最佳 Prompt**** 指令了 並已推上 github Phase 2 Change03 的異動

請幫我驗收 git diff 是否全部 Change03 要求都有通過

是否能推進到 Phase 2 change-04 了呢
顯示更多顯示較少

可以，我已依照你原本 **Change03 最佳 Prompt** 的驗收條件，對 commit **ce914a768d0766ca7d62e18343b5d4f5636182fe** 做了一輪規格層級驗收。

先講結論：

Change03 : opengym-env 的 specification phase 實質上通過，可以推進到 Phase 2 change-04 : drl-mvp proposal / design / specs / tasks。

但仍然一樣：**不批准 /opsx:apply，不開始 implementation，不跑 ns-3，不啟動 ns3-gym，不訓練 DQN / PPO。**

我有一個小但重要的 housekeeping 建議：**tasks.md** 的 final review 11.1、11.2、11.3 目前在 diff 裡仍是未勾選狀態；commit message 雖然寫了「strict validation passed」，但 task checklist 本身尚未同步勾選。因此你可以要求 Antigravity 在下一步前，把 11.1 / 11.2 / 11.3 標註為已完成或附上 validation result；11.4 由你這次驗收後再標註 approved；11.5 必須保持未執行 **/opsx:apply**。commit message 本身顯示 change-03 完成了 4/4 artifacts、8 spec files，且 strict validation passed；diff 也顯示 change-02 的 **/opsx:apply** deferred annotation 已正確保留。

[change-03: opengym-env proposal...](#)

驗收結論

驗收項目	結果	判斷
官方 OpenSpec workflow 是否被要求與記錄	通過	tasks.md 記錄 @fission-ai/openspec 、 openspec list --json 、official OpenSpec change structure、非 fake workflow。 change-03: opengym-env proposal...
是否承接 Change01 / Change02	通過	design 明確承接 project-charter 與 ns3-baseline-benchmark，包含 ns-3.40、NewReno / CUBIC / BBR、metrics、single bottleneck、S1 / S2。 change-03: opengym-env proposal...
是否只做 specification、不做 implementation	通過	proposal / tasks 明確禁止 C++、Python、shell、ns-3 execution、ns3-gym launch、DQN / PPO training。 change-03: opengym-env proposal...
MDP interface 是否完整	通過	已定義 $M = (S, A, P, R, \gamma)$ ，並明確說明 $S/A/P/R/\gamma$ ，transition 由 ns-3 dynamics 決定。 change-03: opengym-env proposal...
Environment boundary 是否清楚	通過	environment = single bottleneck ns-3 simulation 的 RL abstraction；agent 只控制 sender-side abstraction，不控制整個網路，不修改 kernel-level TCP。 change-03: opengym-env proposal...
Observation space 是否清楚	通過	已固定 MVP observation 5 欄位：throughput_norm、delay_norm、loss_norm、congestion_indicator、prev_action_norm，且有 normalization 與 fallback。 change-03: opengym-env proposal...
Action space 是否清楚	通過	已固定 Discrete(3)：decrease / keep / increase，並禁止 continuous action 與 PPO 作為 MVP。 change-03: opengym-env proposal...
Reward 是否符合要求	通過	已定義 multi-objective reward：throughput 正向、delay / loss 懲罰，並使用 λ 避免和 discount factor γ 混淆。 change-03: opengym-env proposal...
Reset / Step / Episode 是否清楚	通過	已定義 reset、step、terminated / truncated、info dict，且對齊 Gymnasium 新版 API 概念。 change-03: opengym-env proposal...
Smoke test 是否清楚	通過	已定義 ST-01 到 ST-10，且明確 smoke test 只驗 environment correctness，不驗 agent performance。 change-03: opengym-env proposal...
Downstream dependency 是否能接 Change04	通過	明確規定 Change04 必須使用 observation/action/reward/episode/smoke test，且不得改成 PPO、continuous action、throughput-only reward。 change-03: opengym-env proposal...
Risk register 是否完整	通過	已包含 observation、RTT、queue、action mapping、reward scale、smoke test、PPO、fake OpenSpec 等風險與 fallback。 change-03: opengym-env proposal...
是否有 CI / workflow run 可佐證	無	GitHub workflow runs 查詢結果為空，代表沒有 Actions workflow run 可作為外部 CI 佐證；本次驗收主要依 commit diff 與 Antigravity 回報的 OpenSpec validation。

重要觀察：OpenSpec 流程有對齊

你這次最擔心的是「Antigravity 是不是真的用官方 OpenSpec，而不是自己假裝一套」。從 commit diff 來看，這次有比前面更明確地把官方 OpenSpec 驗證寫進 **tasks.md**：

- 確認 @fission-ai/openspec 已安裝。
- 確認 `openspec list --json` 可用。
- 確認 Change03 是在 official OpenSpec 底下建立。
- 確認 artifacts 位於 `openspec/changes/opengym-env/`。
- 確認沒有 fake OpenSpec workflow。

[change-03: opengym-env proposal...](#)

這點符合你要求的治理目標。OpenSpec 官方 README 也顯示它的標準工作流是先 `/opsx:propose` 產生 `proposal.md` / `specs` / `design.md` / `tasks.md`，再等後續 `/opsx:apply` 進入實作；也就是說，現在只完成 spec artifacts，而不 apply，是正確狀態。  [GitHub](#)

Change03 要求逐項核對

1. MDP / Environment interface

通過。

設計中已明確定義：

$$M = (S, A, P, R, \gamma)$$

並且把：

- S ：observation space
- A ：Discrete action space
- P ：由 ns-3 simulation 決定
- R ：multi-objective reward
- γ ：discount factor

全部寫清楚。這符合 Phase 2 Change03 的核心：先定義 interface，不進 implementation。

[change-03: opengym-env proposal...](#)

2. Observation space

通過。

MVP observation 被固定成 5 個欄位，且有固定順序：

```
[throughput_norm, delay_norm, loss_norm, congestion_indicator, prev_action_norm]
```

它也把 enhanced observation，例如 queue occupancy、delay gradient、estimated bandwidth、ACK inter-arrival 等全部列為 future extension，不阻塞 MVP。這是正確的，因為 Change04 的 DQN MVP 需要固定 input shape，不能邊訓練邊改 state。

[change-03: opengym-env proposal...](#)

3. Action space

通過。

Action space 明確是：

```
A = {0: decrease, 1: keep, 2: increase}
```

而且有 action safety rules：不得造成負值 rate / cwnd-like variable、action effect bounded、action 必須記錄到 info dict、不直接修改 kernel-level TCP。這符合 DQN MVP 的離散動作需求。Stable-Baselines3 的 DQN

 [stable-](#)

文件顯示 DQN 對應 Discrete action space，因此這個規格是合理的。[baselines3.readthedocs.io](#)

4. Reward design

通過。

Reward 已定義為：

```
r_t = \alpha \cdot throughput\_norm\_t - \beta \cdot delay\_norm\_t - \lambda \cdot loss\_norm\_t
```

並且明確標註：

- $\alpha / \beta / \lambda$ 不在 Change03 固定
- Change04 才定義初始值
- loss penalty 使用 λ ，不使用 γ ，避免和 discount factor 混淆
- throughput-only reward 是 forbidden / failure mode

這一點符合前面 Change02 的 evaluation philosophy：不能只看 throughput，要同時納入 delay / RTT 與 loss。

[change-03: opengym-env proposal...](#)

5. Reset / Step / Gym interface

通過，但有一個形式提醒。

內容已定義 reset / step / terminated / truncated / info dict，並且明確對應 Gymnasium 的新版介面概念。

Gymnasium 官方 Env API 也定義 `step()` 回傳 observation、reward、terminated、truncated、info，

`reset()` 回傳初始 observation 與 info，因此 Change03 的規格方向是正確的。 [Gymnasium](#)

小提醒：`smoke-test.md` 中有一段概念性 pseudo-code。這不是 implementation code，但你後續回饋 Antigravity 時，可以要求他保持註解「conceptual pseudo-code only」，避免 Phase 3 前被誤解成已開始寫 Python。

6. Smoke test

通過。

ST-01 到 ST-10 很完整，包含 reset returns valid observation、random discrete action accepted、reward finite、terminated / truncated defined、info dict required fields、no crash、log format compatible with Change02 metrics、action applied recorded 等。這正好是 Change04 進入 DQN training 前的 gate。

[change-03: opengym-env proposal...](#)

7. Downstream dependency

通過。

Change03 明確約束 Change04：

- observation shape 不得自行更改
- action space 不得改 continuous
- reward 不得 throughput-only
- smoke test 必須先通過
- 不得把 PPO 當 MVP
- 不得加入 IPFS / QUIC / multi-agent / multi-path

這代表它已經可以作為 Change04 的上游規格。

[change-03: opengym-env proposal...](#)

我看到的兩個小問題

小問題 1：tasks.md final review 沒有同步勾選

tasks.md 的 11.1、11.2、11.3 目前仍是未勾選：

- 11.1 `openspec status --change "opengym-env" --json`
- 11.2 `openspec validate "opengym-env" --type change --strict --json`
- 11.3 Submit Change 03 to Spec Owner for review

但 commit message 寫了「strict validation passed」，所以這可能只是 checklist 沒同步更新。

[change-03: opengym-env proposal...](#)

建議處理：

請 Antigravity 在下一個 commit 或你餵回去的 prompt 裡補上：

```
11.1 completed: openspec status confirms isComplete: true
11.2 completed: openspec validate --strict confirms valid: true, issues: []
11.3 completed: submitted to Spec Owner for review
11.4 pending until Spec Owner approval
11.5 remains unchecked: /opsx:apply deferred until Phase 3
```

如果你不想為這個小問題再開 commit，也可以在你回覆 Antigravity 的驗收 prompt 中正式記錄：「內容通過，但 final review checklist 11.1 / 11.2 / 11.3 should be updated for bookkeeping。」

小問題 2：prev_action_norm 一邊說 fixed shape [5]，一邊又說可 fallback 到 [4]

這不是阻斷問題，但要注意。規格目前主體把 observation shape 固定為 [5]，但 fallback rules 說如果 prev_action_norm 有困難可降為 [4]。這是合理 fallback，但 Change04 開始前要做出明確決策：**預設仍以 [5] 為主，除非 smoke test 證明 prev_action_norm 實作困難，才由 Spec Owner 批准降為 [4]。**

[change-03: opengym-env proposal...](#)

這點不影響 Change03 通過，反而是風險控管寫得比較完整；只是 Change04 prompt 要明確要求「default shape = [5]」。

是否能推進到 Phase 2 change-04？

可以，我建議批准推進到 Phase 2 change-04：drl-mvp proposal / design / specs / tasks only。

但批准範圍要寫得很精準：

正式批准：

```
Approved to proceed to Phase 2 change-04:drl-mvp proposal / design / specs / tasks only.
```

仍不批准：

```
Do not run /opsx:apply for Change 03.
Do not start ns-3 coding.
Do not start ns3-gym implementation.
Do not execute smoke test yet.
Do not train DQN.
Do not introduce PPO.
Do not create experiment results.
Do not write analysis scripts.
Do not implement IPFS / QUIC / multi-agent / multi-path.
```

建議餵回 Antigravity 的驗收結論

你可以直接貼這段給 Antigravity：

```
I have reviewed commit ce914a768d0766ca7d62e18343b5d4f5636182fe.
```

Conclusion:

```
Change 03:opengym-env specification phase is approved.
```

The following are accepted:

- proposal.md
- design.md

- tasks.md
- specs/mdp-interface.md
- specs/observation-space.md
- specs/action-space.md
- specs/reward-function.md
- specs/episode-step-reset.md
- specs/smoke-test.md
- specs/env-risk-register.md
- official OpenSpec workflow confirmation
- no implementation included
- no DQN / PPO training included
- no IPFS / QUIC / multi-agent / multi-path scope drift

One bookkeeping note:

tasks.md final review items 11.1 / 11.2 / 11.3 should be marked completed or annotated with the actual OpenSpec status / strict validation result if not already updated.

Item 11.4 may be marked approved after this Spec Owner review.

Item 11.5 must remain not applied:

Do NOT run /opsx:apply for Change 03 until Phase 3 is explicitly approved.

Formal approval:

Approved to proceed to Phase 2 change-04:drl-mvp proposal / design / specs / tasks only.

Not approved:

- Do not run /opsx:apply for Change 03.
- Do not start ns-3 coding.
- Do not start ns3-gym implementation.
- Do not execute smoke test yet.
- Do not train DQN.
- Do not introduce PPO.
- Do not create experiment results.
- Do not write analysis scripts.
- Do not implement IPFS / QUIC / multi-agent / multi-path.

Next step:

Start Phase 2 / OpenSpec change-04:drl-mvp proposal / design / specs / tasks only.

最終判定

Change03 通過。可以進 Change04。

但請維持 Phase 2 原則：

現在只做 OpenSpec 真相來源，不做 implementation。

下一步是生成或執行：

Phase 2 / OpenSpec change-04 : drl-mvp proposal / design / specs / tasks only